Now with 16-QAM modulation :-

parity check matrix

$$H = \begin{matrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{matrix}$$

```
def encode(bits):

while len(bits) % 4 != 0
        bits = np.append(bits, 0)

Output = [ ]

for i in range(0, len(bits), 4):
        d = bits[ i : i+4 ]

P1 = ( d[0] + d[1] + d[2] ) % 2
```

```
P2 = (d[0] + d[1] + d[3]) % 2
P3 = (d[0] + d[2] + d[3]) % 2
output.extend([d[0], d[1], d[2], d[3]
        , P1, P2, P3])
return np.array(output).
```

```
def decode(bits):
while len(bits) % 7 != 0:
    bits = bits[:-1]
    output = []
    For i in range(0, len(bits), 7):
    c = bits[i:i+7].copy().
```

copy to modify we might need original

Original.

$S_1 = (c[0] + c[1] + c[2] + c[4]) \% 2$

$S_2 = (c[0] + c[1] + c[3] + c[5]) \% 2$

$S_3 = (c[0] + c[2] + c[3] + c[6]) \% 2$

if $S_1 == 0$ and $S_2 == 0$ and

$S_3 == 0$.

Output. extend $(c[0:4])$

elif $S_1 == 1$ and $S_2 == 1$ and $S_3 == 1$

$c[0] = 1 - c[0]$

like wise.

$S_1 = 1$ $S_2 = 1$ $S_3 = 0$

$c[1] \Rightarrow$ flip

$S_1 = 1 \quad S_2 = 0 \quad S_3 = 1$

$c[2] \rightarrow$ flip

$S_1 = 0 \quad S_2 = 1 \quad S_3 = 1$

$c[3] \rightarrow$ flip

Output extend $\in [0:4]$

Now modulation:-

```
def modulate(bits):-

while len(bits) % 4 != 0:

    bits = np.append(bits, 0).

    Symbols = []

For i in range(0, len(bits), 4):
```

$b = \{i : i+u\}$

mapping rules are

00	-3
01	-1
11	1
10	3

add if else statements.

put two no. I - real

Q - imaginary.

So

Symbols.append(I + 1j Q)

return np.array(symbols / np.sqrt(20))

```
def tx. fiter (symbols):
    L = usf_filter    L = 8.

upsampled = np.zeros (len(symbols)
L , dtype = complex )
upsampled [::2] = symbols.
filter_len = 16
h = np. ones (filter_len / filter_len)
```

every symbols at 8th position

pividing by 16 each = 0.062s.

low-pass filter.

lets slow changes pass and

blocks fast changes.

filtered = np.convolve (upsampled, h, mode = 'same')

Signal : (A, 0 0 0 0 , 0 B 0, ...)

filter : {1/16, 1/16, ..., 1/16}

output = $\left(\frac{A \times 1}{16}, \frac{0 \times 1}{16}, \ldots\right)$

$A/16$ = very small.

return filtered / np.sqrt (np.mean (np.abs filtered) ** 2)).

Normalise power

rx_filter(signal) :-

1 - usf filter

filter_den = 16

h = np.once(filter_den) / filter_den

Creates identical filter to transmitter.

Called matched filter - gives best possible recovery.

filtered = np.convolve (signal, h, mode = 'same').

delay = filter_den // 2

added delay

symbols = filtered [delay ::2]

return symbols

BIT Error rate.

```
def BER (original, received)

min_len = min (len original) len (received)

errors = np.sum (original[: minlen] != received[: min_len])

return errors / min_len
```